

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

Asignatura: **Estructura y Programación de Computadoras**

Grupo: 05

DISEÑO E IMPLEMENTACIÓN DE UN ENSAMBLADOR Y ENLAZADOR MODULAR

Desarrollo de un Toolchain Completo de Software de Sistema para la Arquitectura Intel IA-32 utilizando el Lenguaje C Estándar

PROFESORA:

- Ing. Mercado Martinez Ariel Adara

FECHA DE ENTREGA:

- 11 de Junio de 2026

EQUIPO DE TRABAJO:

- Carbajal Lopez Jose David
- Laguna Eslava Yael Xanthe
- Prieto Tirado Andrea Estefani
- Sánchez López Edmar Alejandro

1. Resumen Ejecutivo

El presente reporte técnico documenta de forma exhaustiva el diseño arquitectónico, los fundamentos lógicos y la implementación práctica de una cadena completa de traducción de software (*toolchain*) integrada por un Ensamblador de dos pasadas y un Enlazador dinámico-estático (Linker). El sistema completo fue estructurado bajo las directrices estrictas del estándar de programación ANSI C / C11, concibiendo una herramienta de bajo nivel altamente eficiente y determinista, capaz de transformar código fuente simbólico de bajo nivel en estructuras empaquetadas de código máquina sin depender de entornos virtuales o de un recolector de basura (*garbage collector*) automático que altere el tiempo de ejecución.

El desarrollo del proyecto aborda directamente las restricciones físicas del hardware y el modelo de ejecución CISC característico de la familia Intel de 32 bits. La solución gestiona de manera rigurosa buffers fijos de memoria lineal, estructuras de registros interconectadas y arreglos crudos de bytes mutables. El pipeline de procesamiento se activa mediante la lectura sistemática de archivos de texto con mnemónicos tipo NASM. La primera pasada del ensamblador opera un análisis léxico-sintáctico mediante el desglose de cadenas por punteros nativos de tipo char*, construyendo una Tabla de Símbolos estructurada encargada de computar de forma secuencial y exacta la evolución del Contador de Programa (PC) simulado en la sección de texto (.text). Este cálculo algorítmico determina el tamaño exacto que ocupará cada instrucción de acuerdo con la complejidad de sus operandos y direccionamientos físicos.

Durante la segunda pasada, el sistema realiza tareas de codificación pura y síntesis matemática. El motor traduce las instrucciones mnemónicas mapeando opcodes base, calcula desplazamientos de salto relativo adelantados y encripta la configuración física de los registros mediante el empaquetado exacto de los campos ModRM y de los bytes SIB (Escala-Índice-Base). Finalmente, el Enlazador consolida el ciclo de vida del software fusionando bloques binarios provenientes de múltiples módulos u objetos intermedios. El Linker barre los símbolos declarados como globales externos, recalcula y ajusta los desplazamientos físicos absolutos en las rutinas concatenadas e inyecta los offsets de control reubicados bajo el formato Little-Endian. El resultado es un entorno cohesivo, defensivo ante entradas de sintaxis corruptas y con una salida binaria transparente y fiel al funcionamiento del silicio Intel IA-32.

2. Arquitectura General del Sistema

La arquitectura general de la herramienta de software se rige bajo los conceptos fundamentales de alta cohesión y acoplamiento débil (*Loose Coupling*). El procesamiento de un archivo fuente de ensamblador se subdivide de manera formal en un pipeline secuencial de fases acopladas mediante estructuras fijas de intercomunicación.

2.1. Componentes del Código Fuente y Estructura Detallada de Archivos

- ***estructuras.h (Cabecera Global y Tipos Base)***: Este archivo actúa como el núcleo de tipos de datos del compilador. Contiene la enumeración formal de los componentes léxicos mapeados por el autómata (TipoToken), permitiendo una categorización rígida de los elementos del lenguaje como directivas, mnemónicos de instrucción, registros físicos de 32 bits, números inmediatos o símbolos delimitadores. Asimismo, define las estructuras esenciales de datos como *Token* (que empareja el tipo con un buffer de texto fijo de 32 bytes) y *Símbolo*, una estructura optimizada que almacena el identificador alfanumérico de las etiquetas del código junto con su dirección de offset de 32 bits y sus propiedades de visibilidad (local o global). Al centralizar estos tipos, se previene la duplicación de código y se garantiza la consistencia del alineamiento en memoria.
- ***main.c (Orquestador Central y Ciclo de Vida)***: Es la rutina principal y punto de entrada que administra la ejecución completa de la cadena de herramientas (*toolchain*). Utiliza las llamadas estándar de bajo nivel como ``fopen`` y ``fgets`` para realizar un escaneo por flujos del archivo con extensión ``.asm`` suministrado a través de los argumentos de la línea de comandos (``argc`` y ``argv``). Su responsabilidad es coordinar de manera secuencial la ejecución de los módulos, verificar que cada fase concluya sin errores de segmento, alimentar las pasadas del codificador compartiendo variables globales mapeadas con almacenamiento externo (``extern``) y proveer la telemetría detallada del consumo de memoria y tamaño binario en la terminal del sistema.
- ***lexer.c (Analizador Léxico por Autómata de Cadena)***: Módulo encargado del análisis léxico y de la segmentación de cadenas de texto. A diferencia de enfoques destructivos tradicionales (como el uso de la función `strtok`), este módulo implementa un pequeño autómata finito que escanea el buffer de entrada carácter por carácter. Este enfoque algorítmico es estrictamente necesario para poder aislar, detectar y clasificar correctamente los símbolos matemáticos y delimitadores requeridos por el direccionamiento complejo SIB (tales como `[`, `]`, `+` y `*`), permitiendo separar operandos que no contienen espacios en blanco sin alterar la integridad de la cadena original.

Cada token procesado se normaliza a mayúsculas y se inyecta de manera secuencial en un arreglo global que alimenta la fase sintáctica.

- ***parser.c (Analizador Sintáctico y Cálculo de Direcciones - Pasada 1):*** Este componente opera la primera pasada formal del ensamblador. Su objetivo prioritario es mantener un Contador de Programa (PC) dinámico para determinar el tamaño exacto en bytes de cada instrucción en el binario final. Además de calcular heurísticamente los desplazamientos para Opcodes, bytes ModRM y bytes SIB en la sección de código (.text), el analizador incorpora el manejo robusto de directivas de memoria. Es capaz de procesar declaraciones como DB, DW, DD, RESB y RESW, reservando algebraicamente las cantidades exactas de memoria. De igual manera, procesa directivas de alteración del origen del programa (ORG) e ignora de forma transparente las directivas exclusivas de alcance del enlazador (GLOBAL y EXTERN), registrando exitosamente las etiquetas en la Tabla de Símbolos global.
- ***encoder.c (Codificador Matemático y Emisión Hexadecimal - Pasada 2):*** Representa la fase de síntesis binaria. Su tarea es la traducción directa de mnemónicos y operandos a su representación hexadecimal. Cuando procesa un direccionamiento indexado complejo (por ejemplo, [EBX+ECX*4+8]), el codificador extrae matemáticamente la escala, el índice y la base para ensamblar a nivel de bits el byte de Modo de Direccionamiento (ModRM) y el byte SIB, aplicando desplazamientos binarios (bit-shifting). Adicionalmente, el codificador implementa el soporte avanzado para la familia de operaciones aritméticas unarias y complejas (tales como MUL, DIV, NOT, NEG, IMUL, IDIV) haciendo uso algorítmico del sistema de extensión de Opcodes (Grupo F7) estandarizado por Intel.
- ***linker.c (Enlazador de Software y Reubicación de Símbolos):*** El enlazador actúa en la etapa final del ciclo de vida del software. Para garantizar la modularidad, se diseñó una estructura de datos formal denominada CabeceraObjeto (que encapsula una firma de validación "OBJ1", el tamaño de la sección de texto y la cantidad de símbolos exportados). Al finalizar la segunda pasada, el sistema serializa la tabla de símbolos y el código máquina, escribiéndolos en el disco bajo un formato intermedio de archivo objeto (salida.obj). Finalmente, la rutina del Linker abre estas estructuras físicas, extrae los flujos binarios, resuelve las referencias cruzadas y emite un archivo ejecutable binario plano (programa.bin) listo para su distribución.

3. Evidencia de Ejecución y Casos de Prueba

Para validar la robustez y precisión algorítmica del desarrollo en C, se sometió el sistema completo a una prueba de compilación de archivos externos con direccionamiento complejo.

3.1. Simulación Léxica e Instrucciones Evaluadas

```
1  SECTION .data
2      variable_memoria DD 100      ; Validar directiva DD (4 bytes)
3      buffer_vacio RESB 16         ; Validar directiva RESB (16 bytes)
4
5  SECTION .text
6  GLOBAL main
7
8  main:
9      MOV EAX, 10                  ; Movimiento de Inmediato
10     MOV EBX, 5
11     SUB EAX, EBX                  ; Aritmética Registro-Registro
12     ADD ECX, 2                    ; Aritmética Inmediata
13
14     MOV EAX, [EBX+ECX*4+8]        ; PRUEBA DE FUEGO: Byte ModRM y Byte SIB + Disp8
15
16     CMP EAX, 0
17     JE fin                        ; Salto Condicional (Referencia adelantada)
18
19     JMP main                      ; Salto Absoluto
20
21 fin:
22     RET                          ; Retorno de sistema
```

3.2. Telemetría de la Consola de Comandos (Caso Exitoso)

```
[INFO] Compilando Ensamblador en C nativo...
=====

[RUN] Corriendo prueba avanzada...
-----

--- ENSAMBLADOR IA-32 Y LINKER ---
[1] Ejecutando Pasada 1 (Tabla de Simbolos y Memoria)...
[2] Ejecutando Pasada 2 (Generacion Opcodes/ModRM/SIB)...
[3] Creando Modulos (Formato Objeto)...
[EXIT0] Archivo objeto 'salida.obj' generado correctamente (25 bytes de texto).
[4] Ejecutando Linker...
[EXIT0] Linker completado. Binario 'programa.bin' generado.
--- PROCESO FINALIZADO EXITOSAMENTE ---
```

4. Manejo Preventivo de Errores (Caso Fallido de Entrada)

Para asegurar el cumplimiento estricto de las reglas del proyecto, se desarrolló una suite automatizada de pruebas de integración (Caja Negra) ejecutada mediante rutinas Batch y código envolvente en C (`test_runner.c`). Esta suite evolucionó de la simple detección de caracteres inválidos hacia la evaluación de la resiliencia del pipeline completo frente a direccionamientos SIB complejos y la resolución de módulos externos (EXTERN).

El sistema actual incorpora un mecanismo defensivo robusto mediante un autómata de estados finitos en el analizador léxico. Este diseño garantiza que el ensamblador no sufra desbordamientos de buffer (buffer overflows) al leer operandos concatenados sin espacios, ni colapse al encontrar símbolos desconocidos. En caso de fallos críticos, como la ausencia de archivos fuente o la imposibilidad de manipular flujos de salida binaria, los módulos retornan códigos de error seguros al sistema operativo (errorlevel), deteniendo el proceso de traducción antes de generar binarios corruptos y protegiendo la estabilidad del entorno de ejecución.

5. Conclusiones Generales

El proceso de diseño, estructuración e implementación de este traductor modular empleando el lenguaje C estándar ha representado una experiencia fundamental dentro del desarrollo académico de la ingeniería de sistemas. Al prescindir por completo de capas automáticas de abstracción y recolectores de basura automáticos de alto nivel, el equipo se vio obligado a asumir el control absoluto sobre el mapa de memoria y la manipulación de datos a bajo nivel. Esta exigencia conllevó una comprensión profunda del manejo de buffers fijos, la aritmética de punteros en la manipulación de cadenas y el riesgo latente de desbordamientos de pila o fallos de segmento (*segmentation faults*), los cuales fueron mitigados mediante un diseño defensivo robusto y estructurado.

En el aspecto puramente teórico de la arquitectura de computadoras, el desarrollo de la lógica del ensamblador y el enlazador permitió asimilar de forma práctica la complejidad de la arquitectura nativa Intel IA-32 de 32 bits. La traducción manual de instrucciones mnemónicas a sus códigos de operación (*opcodes*) correspondientes arrojó luz sobre los mecanismos físicos que realiza el procesador durante el ciclo de búsqueda y ejecución. Particularmente, la decodificación y el empaquetado matemático de los bytes ModRM y SIB evidenciaron cómo las instrucciones complejas con direccionamiento indexado y desplazamientos numéricos se configuran a nivel de registros lógicos. Asimismo, el cálculo matemático y preciso del

Contador de Programa (PC) para la resolución de saltos relativos hacia adelante y hacia atrás demostró la importancia crítica de la primera pasada del ensamblador.

Por último, la exitosa consolidación del módulo enlazador expandió de gran manera la perspectiva conceptual sobre el software de base. Comprender que un programa ejecutable moderno no es una entidad atómica y aislada, sino el resultado final de la fusión armónica de múltiples módulos de código independientes proporcionó al equipo una visión integral del ciclo de vida del software. La implementación de algoritmos de reubicación de símbolos globales y la escritura controlada de bytes en formato *Little-Endian* garantizaron que las referencias externas cruzadas se entrelazaran de manera exacta. Este proyecto no solo valida las competencias técnicas del equipo en programación de sistemas, sino que establece bases firmes para el estudio avanzado de la ingeniería de compiladores y la arquitectura de sistemas computacionales contemporáneos.

6. Referencias Bibliograficas

1. *Intel Corporation. Intel® 64 and IA-32 Architectures Software Developer's Manual. Volume 2 (2A, 2B, 2C & 2D): Instruction Set Reference. Edición Oficial 2024.*
2. *Kernighan, B. W., & Ritchie, D. M. The C Programming Language (2nd Edition). Prentice Hall. Referencia formal para el manejo de punteros y estructuras de datos estáticas.*
3. *NASM Development Team. The Netwide Assembler: Official Documentation and Directive Guide. Manual técnico de sintaxis para compatibilidad de mnemónicos de bajo nivel.*